

TP2 - Requests Tutorial

Python Avance - 2SE

Annee universitaire: 2025/2026

Auteur: Wahid Hamdi

Page de garde

****TP2 - Requests Tutorial****

****Matière : ** Python Avancé**

****Année universitaire : ** 2025/2026**

****Auteur : ** Wahid Hamdi**

Introduction

Le module **Requests** est une bibliothèque HTTP élégante et simple pour Python, conçue pour être utilisée par des humains. Elle permet d'effectuer des requêtes HTTP pour interagir avec n'importe quel site web ou API directement depuis une application Python.

Ce TP explore les fonctionnalités essentielles de la bibliothèque Requests : les requêtes GET et POST, les codes de statut HTTP, la gestion des erreurs, les timeouts, les en-têtes HTTP, le web scraping avec BeautifulSoup, et une comparaison avec le module urllib.

Objectifs du TP

- Comprendre et utiliser le module Requests pour effectuer des requêtes HTTP
- Savoir interpréter les codes de statut HTTP
- Apprendre à envoyer des données avec une requête POST
- Gérer les erreurs et les timeouts
- Utiliser les en-têtes HTTP pour l'authentification
- Découvrir le web scraping avec BeautifulSoup
- Comparer Requests avec urllib

Travail demandé

1. **GET Request** : Effectuer une requête GET simple vers un site web
2. **HTTP Status Codes** : Comprendre et afficher les codes de statut HTTP
3. **Request Content** : Accéder au contenu de la réponse
4. **POST Request** : Envoyer des données via une requête POST
5. **Handling Errors** : Gérer les erreurs HTTP
6. **Setting a Timeout** : Définir un timeout pour les requêtes
7. **HTTP Request Headers** : Utiliser des en-têtes HTTP personnalisés
8. **Web Scraping with BeautifulSoup** : Extraire des données d'une page web
9. **Requests vs urllib** : Comparer Requests avec le module urllib

Réalisation étape par étape

1. GET Request

Un script simple effectue une requête GET vers `https://www.example.com` et affiche l'objet response ainsi que le code de statut.

```
import requests

url = "https://www.example.com"
response = requests.get(url)
print(response)
print(f"Status code: {response.status_code}")
```

****Résultat :**** L'objet response affiché montre `

2. HTTP Status Codes

Ce script teste différents codes de statut HTTP :

- Un code 200 (Succès) avec une requête vers `https://www.example.com`
- Un code 404 (Non trouvé) avec une requête vers `https://httpbin.org/status/404`

```
response = requests.get("https://www.example.com")
print(f"Success (200): {response.status_code}")

response = requests.get("https://httpbin.org/status/404")
print(f"Not Found (404): {response.status_code}")
```

****Résultat :**** Les codes 200 et 404 sont affichés correctement.

3. Request Content

Le script accède au contenu brut de la réponse via l'attribut `content` et affiche les 500 premiers caractères.

```
response = requests.get("https://www.example.com")
content = response.content
print(content[:500])
```

****Résultat :**** Le contenu HTML de la page est affiché (les 500 premiers octets).

4. POST Request

Envoi de données JSON via une requête POST vers `https://httpbin.org/post`, un service de test HTTP.

```
data = {"name": "Salah", "message": "Hello!"}
url = "https://httpbin.org/post"
response = requests.post(url, json=data)
response_data = response.json()
print(response_data)
```

****Résultat :**** Le serveur renvoie un dictionnaire contenant les données envoyées, confirmant la réception.

5. Handling Errors

Vérification du code de statut pour détecter les erreurs HTTP.

```
response = requests.get("https://httpbin.org/status/404")
if response.status_code != 200:
    print(f"HTTP Error: {response.status_code}")
```

****Résultat :**** Le message `HTTP Error: 404` s'affiche lorsque le serveur renvoie une erreur.

6. Setting a Timeout

Définition d'un timeout de 5 secondes pour une requête vers un endpoint qui prend 10 secondes à répondre.

```
url = "https://httpbin.org/delay/10"
try:
    response = requests.get(url, timeout=5)
except requests.exceptions.Timeout as err:
    print(err)
```

****Résultat :**** Une exception `Timeout` est levée et capturée, démontrant l'importance de définir un timeout.

7. HTTP Request Headers

Envoi d'un en-tête d'autorisation avec un token Bearer.

```
auth_token = "XXXXXXXXX"
headers = {"Authorization": f"Bearer {auth_token}"}
url = "https://httpbin.org/headers"
response = requests.get(url, headers=headers)
print(response.json())
```

****Résultat :**** Le serveur renvoie les en-têtes reçus, incluant l'en-tête Authorization.

8. Web Scraping avec BeautifulSoup

Extraction du titre, du premier paragraphe et des liens de `https://www.example.com` en utilisant BeautifulSoup.

```
from bs4 import BeautifulSoup

url = "https://www.example.com"
response = requests.get(url)
soup = BeautifulSoup(response.content, "html.parser")

title = soup.title.text
content = soup.find("p").text
links = [a["href"] for a in soup.find_all("a")]
```

****Résultat :**** Le titre "Example Domain", le texte du paragraphe et le lien sont extraits et affichés.

9. Requests vs urllib

Comparaison avec urllib (module intégré de Python) pour une requête POST.

```
import urllib.request
import urllib.parse

data = urllib.parse.urlencode({"key": "value"}).encode("utf-8")
req = urllib.request.Request("https://www.example.com", data=data, method="POST")
with urllib.request.urlopen(req) as response:
    html = response.read().decode("utf-8")
```

****Résultat :**** La syntaxe est plus verbeuse qu'avec Requests, confirmant la simplicité de ce dernier.

Résultats obtenus

Tous les scripts ont été exécutés avec succès :

Exercice	Statut
GET Request	Reussi
HTTP Status Codes	Reussi
Request Content	Reussi
POST Request	Reussi
Handling Errors	Reussi
Setting a Timeout	Reussi (exception capturée)
HTTP Request Headers	Reussi
Web Scraping with BeautifulSoup	Reussi
Requests vs urllib	Reussi

Le module Requests offre une interface beaucoup plus simple et intuitive que urllib pour effectuer des requêtes HTTP. Les principales différences observées sont :

- **Requests** : syntaxe concise, gestion automatique des JSON, documentation claire
- **urllib** : plus verbeux, nécessite plus de code pour des opérations simples

Conclusion

Ce TP a permis de maîtriser les fondamentaux du module Requests en Python :

1. **Requêtes GET et POST** : Les méthodes de base pour interagir avec des ressources web
2. **Codes de statut HTTP** : Compréhension des différentes catégories (2XX succès, 3XX redirection, 4XX erreur client, 5XX erreur serveur)
3. **Gestion des erreurs** : Importance de vérifier les codes de statut et de gérer les timeouts
4. **Headers HTTP** : Utilisation pour l'authentification et la personnalisation des requêtes
5. **Web scraping** : Combinaison de Requests et BeautifulSoup pour l'extraction de données web

Requests se distingue comme la bibliothèque HTTP de choix pour Python, offrant simplicité, élégance et puissance dans un seul package.